ETHCF Swapboard Audit Report

Prepared by Cyfrin

Version 2.0

Lead Auditors

qpzm

Fresco

Ironside

Assisting Auditors

Dacian (Gas Optimization)

March 4, 2026

Contents

1 About Cyfrin **2**

2 Disclaimer **2**

3 Risk Classification **2**

4 Protocol Summary **2**

5 Audit Scope **3**

6 Executive Summary **3**
6.1 Blacklistable Tokens . . . 3
6.2 Rebasing Tokens . . . 3
6.3 ERC20 Integration Analysis . . . 4

7 Findings **6**
7.1 Low Risk . . . 6
7.1.1 No order expiration — stale orders become free options for takers . . . 6
7.1.2 No fill deadline parameter exposes takers to stale transaction execution . . . 7
7.2 Informational . . . 8
7.2.1 Address-based token identity checks can be bypassed by alias tokens and native ERC20 mirrors . . . 8
7.2.2 Use named mappings parameters to explicitly indicate the purpose of keys and values . . . 8
7.2.3 Use read-then-increment in `Swapboard::createOrder, createOrderWithEth` . . . 9
7.3 Gas Optimization . . . 10
7.3.1 Use `ReentrancyGuardTransient` instead of `ReentrancyGuard` for faster `nonReentrant` modifiers . . . 10
7.3.2 Don't initialize `received` to default value in `Swapboard::createOrder` . . . 10
7.3.3 Cache storage to prevent identical storage reads . . . 10
7.3.4 When sending ETH, use `SafeTransferLib::safeTransferETH` instead Of Solidity `call` . . . 10
7.3.5 Don't cache `calldata` array length in `Swapboard::getOrders` . . . 11
7.3.6 Better storage packing in `struct Order` reduces gas cost of all `create, fill, cancel` operations . . . 11

1 About Cyfrin

Cyfrin is a Web3 security company dedicated to bringing industry-leading protection and education to our partners and their projects. Our goal is to create a safe, reliable, and transparent environment for everyone in Web3 and DeFi. Learn more about us at cyfrin.io.

2 Disclaimer

The Cyfrin team makes every effort to find as many vulnerabilities in the code as possible in the given time but holds no responsibility for the findings in this document. A security audit by the team does not endorse the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the solidity implementation of the contracts.

3 Risk Classification

	Impact: High	**Impact: Medium**	**Impact: Low**
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

4 Protocol Summary

`Swapboard` is a trustless OTC bulletin board for ERC20 token swaps on Ethereum. It implements a simple on-chain orderbook where makers deposit tokens into escrow and takers atomically fill orders by providing the requested counterpart tokens.

Key design properties:

- **No admin functions, fees, or upgrades** — the contract is fully immutable and permissionless with no privileged roles
- **Atomic all-or-nothing fills** — orders cannot be partially filled; each fill transfers the full amounts between maker and taker in a single transaction
- **Maker-escrowed** `tokenA` — when creating an order, the maker's selling token (`tokenA`) is transferred into the contract and held in escrow until the order is filled or cancelled
- **Direct taker-to-maker** `tokenB` — when filling an order, the buying token (`tokenB`) is transferred directly from the taker to the maker without passing through the contract
- **Fee-on-transfer rejection for** `tokenA` — the contract uses a balance-before/after check to reject fee-on-transfer tokens as the selling token, preventing accounting mismatches in escrow
- **Native ETH convenience wrappers** — the contract provides `createOrderWithEth`, `fillOrderWithEth`, `cancelOrderUnwrap`, and `fillOrderUnwrap` functions that automatically wrap/unwrap ETH via the canonical `WETH` contract, allowing users to trade native ETH without manually interacting with `WETH`
- **Reentrancy protection** — all state-changing functions use OpenZeppelin's `ReentrancyGuard` with the `nonReentrant` modifier
- **Order lifecycle** — each order transitions from active to inactive exactly once, either through filling or cancellation; only the maker can cancel their own orders

5 Audit Scope

The audit scope was limited to:

```
contracts/src/Swapboard.sol
```

6 Executive Summary

Over the course of 1 days, the Cyfrin team conducted an audit on the Swapboard smart contracts provided by ETHCF. In this period, a total of 11 issues were found.

The findings consist of 2 Low severity issues with the remainder being informational and gas optimizations.

`Swapboard` is a simple, well-structured contract with no admin functions, fees, or upgradeability. The core swap logic is sound — reentrancy is properly mitigated, fee-on-transfer tokens are rejected for the escrowed selling token, and order state transitions are atomic and final. However, due to the permissionless escrow design, certain non-standard ERC20 token behaviors create risks that users must understand before trading.

As part of the audit we wrote additional invariant and halmos verification tests which we provided as an additional deliverable.

6.1 Blacklistable Tokens

Tokens with admin-controlled blocklists (USDC, USDT) pose a fund-locking risk. When a maker creates an order with a blacklistable token as `tokenA` (the escrowed selling token), the escrowed funds become permanently unrecoverable if both the fill and cancel transfer paths are blocked. Specific scenarios include:

- **Maker blocklisted on** `tokenA`: `cancelOrder` transfers `tokenA` back to the maker. If the maker's address is blocklisted on `tokenA` after order creation, the cancel reverts. Fills may still succeed (`tokenA` goes to the taker), but if no taker fills the order, the funds are permanently locked
- **Contract address blocklisted on** `tokenA`: If the `Swapboard` contract address itself is added to a token's blocklist, all outbound transfers of that token revert. Every order escrowing that token becomes unrecoverable — neither fills nor cancellations can move the tokens out
- **Maker blocklisted on** `tokenB`: `fillOrder` transfers `tokenB` directly from taker to maker. If the maker is blocklisted on `tokenB`, all fill attempts revert. The maker can still cancel and recover `tokenA`, but the order is effectively unfillable

Since `Swapboard` has no admin functions or emergency withdrawal mechanism, there is no recovery path once both the fill and cancel transfer paths are blocked for an order.

6.2 Rebasing Tokens

Rebasing tokens (e.g., stETH) change holder balances outside of transfers, which conflicts with `Swapboard`'s fixed-amount escrow accounting:

- **Positive rebase**: The contract's actual token balance increases, but orders still record and transfer the original fixed `amountA`. The excess yield from rebasing accumulates in the contract with no mechanism to recover it, resulting in permanently locked value
- **Negative rebase**: The contract's actual token balance decreases below the recorded `amountA`. Both `fillOrder` (transferring `amountA` to taker) and `cancelOrder` (transferring `amountA` back to maker) revert due to insufficient balance, permanently locking whatever remains
- **Multiple orders with rebasing** `tokenA`: When multiple orders escrow the same rebasing token, fills are served first-come-first-served from the shared balance. After a negative rebase, earlier fills may succeed while later orders become unfillable, creating an unfair distribution of losses

6.3 ERC20 Integration Analysis

A comprehensive review of `Swapboard` against 23 known non-standard ERC20 behaviors (cross-referenced with weird-erc20) confirms the core integration is sound for standard-compliant tokens. Key strengths:

- **Reentrancy (ERC777)**: `nonReentrant` on all mutating functions blocks reentrancy via `tokensToSend` / `tokensReceived` hooks. CEI pattern ensures state consistency during callbacks
- **Missing return values (USDT)**: `SafeERC20` handles non-standard returns across all transfer calls
- **Zero-value transfers**: `amountA` and `amountB` validated non-zero at creation, preventing reverts from tokens like LEND
- **Flash-mintable tokens (DAI)**: Flash-minted tokens deposited via `createOrder` can't be repaid (they're in Swapboard's custody), so the flash-mint transaction reverts entirely

Remaining gaps are inherent to the permissionless escrow design:

- **Fee-on-transfer tokenB (STA, PAXG)**: tokenA is checked via balance-diff, but tokenB is not — the maker may receive less than `amountB`. The team acknowledges this as maker's risk (L101) and has tests covering this scenario
- **Upgradable tokens (USDC, USDT)**: Token logic can change between order creation and fill/cancel (e.g., fee mechanism activated, transfer restrictions added). The team has tests covering the scenario where a token upgrades to introduce fee-on-transfer behavior after order creation
- **Multiple token addresses (proxy)**: Proxied tokens with alternate entry points could bypass the `tokenA == tokenB` check if the same underlying token has two addresses. Unlikely in practice
- **Low decimals (GUSD=2)**: No minimum order size enforced, and precision loss is more significant with fewer decimals
- **ERC-20 native currency (Celo, Polygon)**: If deployed on a chain where the native currency is also an ERC20 and WETH points to the same token, ETH functions could double-count. Safe if the constructor WETH address is correctly set for the target chain

Summary

Project Name	Swapboard
Repository	swapboard
Commit	0c0a7eea363f...
Fix Commit	82f9b4902ae5...
Audit Timeline	Feb 25th - Feb 25th, 2026
Methods	Manual Review

Issues Found

Critical Risk	0
High Risk	0
Medium Risk	0
Low Risk	2
Informational	3
Gas Optimizations	6
Total Issues	11

Summary of Findings

[L-1] No order expiration — stale orders become free options for takers	Acknowledged
[L-2] No fill deadline parameter exposes takers to stale transaction execution	Resolved
[I-1] Address-based token identity checks can be bypassed by alias tokens and native ERC20 mirrors	Acknowledged
[I-2] Use named mappings parameters to explicitly indicate the purpose of keys and values	Resolved
[I-3] Use read-then-increment in `Swapboard::createOrder, createOrderWithEth`	Resolved
[G-1] Use `ReentrancyGuardTransient` instead of `ReentrancyGuard` for faster `nonReentrant` modifiers	Resolved
[G-2] Don't initialize `received` to default value in `Swapboard::createOrder`	Resolved
[G-3] Cache storage to prevent identical storage reads	Resolved
[G-4] When sending ETH, use `SafeTransferLib::safeTransferETH` instead Of Solidity `call`	Resolved
[G-5] Don't cache `calldata` array length in `Swapboard::getOrders`	Resolved
[G-6] Better storage packing in `struct Order` reduces gas cost of all `create, fill, cancel` operations	Resolved

7 Findings

7.1 Low Risk

7.1.1 No order expiration — stale orders become free options for takers

Description: `Swapboard::createOrder` and `Swapboard::createOrderWithEth` create orders with no expiration timestamp. Once created, an order remains fillable indefinitely until explicitly cancelled. This creates a free option problem.

- The `Order` struct has no `deadline` field
- `Swapboard::fillOrder`, `Swapboard::fillOrderWithEth`, and `Swapboard::fillOrderUnwrap` enforce no time-based check
- The subgraph tracks `createdAt` but the contract enforces no deadline
- The frontend shows when an order was created but cannot warn about stale orders

Scenario:

1. Maker creates an order selling 1 WETH for 2,000 USDC when ETH is at $2,000
2. ETH price rises to $4,000 over the next month
3. The order is still active — any taker can buy 1 WETH for 2,000 USDC (50% below market)
4. The maker must actively monitor and cancel, but if they lose wallet access, go offline, or miss the price movement, the order is a sitting target

From the taker's perspective, every open order is a **free call option**: they can wait and only execute when the market moves in their favor, at no cost. In traditional OTC and DeFi limit-order systems (e.g., 0x, CoW Protocol, 1inch Limit Orders), orders include a `deadline` or `expiry` parameter to mitigate this.

Impact: Makers suffer economic loss from stale orders being filled at outdated prices. The risk scales with market volatility and the duration orders remain open.

Proof of Concept:

- Any active order can be filled at any time regardless of how old it is
- No on-chain mechanism to auto-expire orders
- Maker's only option is to manually cancel, which requires wallet access and gas

Recommended Mitigation: Add an optional `deadline` parameter to `createOrder` and `createOrderWithEth`:

```
struct Order {
    address maker;
    address tokenA;
    uint256 amountA;
    address tokenB;
    uint256 amountB;
    uint256 deadline;   // 0 = no expiry
    bool active;
}
```

Then in [`Swapboard::fillOrder`] (and all fill variants):

```
if (order.deadline != 0 && block.timestamp > order.deadline) revert OrderExpired(orderId);
```

This gives makers opt-in time-bounding while preserving backwards compatibility (deadline=0 means no expiry).

Ethcf: Acknowledged; this is a design choice.

7.1.2 No fill deadline parameter exposes takers to stale transaction execution

Description: `Swapboard::fillOrder`, `Swapboard::fillOrderWithEth`, and `Swapboard::fillOrderUnwrap` accept no deadline parameter. In a congested network, a taker's fill transaction could sit in the mempool for extended periods and execute when conditions have changed (e.g., the token price has moved significantly).

This is distinct from M-01 (maker-side order expiry) — this is **taker-side** protection. Even if the order itself has no expiry, the taker should be able to specify "execute my fill only if it happens within N seconds."

This is standard practice in DeFi swap protocols. For example, Uniswap V2 Router02 includes a `deadline` parameter on all swap functions:

```
// Uniswap V2 Router02
function swapExactTokensForTokens(
    uint amountIn,
    uint amountOutMin,
    address[] calldata path,
    address to,
    uint deadline    // UNIX timestamp
) external returns (uint[] memory amounts);
```

The `deadline` prevents miners/block builders from holding a transaction and executing it later at a more favorable price. Uniswap V2, V3, and V4 all include it.

Impact: Takers may have their fill transactions execute at an outdated and unfavorable rate after sitting in the mempool during network congestion.

Proof of Concept:

- Taker submits `fillOrder(orderId)` during congestion
- Transaction sits in mempool for hours
- Token price moves significantly
- Transaction finally executes at the now-unfavorable rate
- Taker had no way to set a deadline to prevent this

Recommended Mitigation: Add a `deadline` parameter to fill functions:

```
function fillOrder(uint256 orderId, uint256 deadline) external nonReentrant {
    if (deadline != 0 && block.timestamp > deadline) revert DeadlineExpired();
    // ...
}
```

Ethcf:

Fixed in commit 572f3c5. Added `uint256 deadline` parameter to `fillOrder, fillOrderWithEth, and fillOrderUnwrap`. When `deadline == 0`, no expiry is enforced. When `deadline != 0 && block.timestamp > deadline`, the call reverts with `DeadlineExpired`.

Cyfrin: Verified. Deadline check is correct — early revert before storage reads, `deadline == 0` allows takers to opt out of deadline enforcement.

7.2 Informational

7.2.1 Address-based token identity checks can be bypassed by alias tokens and native ERC20 mirrors

Description: The protocol treats token identity as address equality (`tokenA == tokenB` and `tokenB == weth`). This assumption breaks when one economic asset has multiple valid addresses (proxy aliases/wrapper aliases) or when a chain exposes the native gas token as an ERC20.

Impact: Informational. Enabling orders that are "same asset, different address".

Proof of Concept: In `createOrder`, only `if (tokenA == tokenB) revert SameToken();` is enforced. In `createOrderWithEth`, only `if (tokenB == weth) revert SameToken();` is enforced. Both checks are address-only, so alias contracts for the same underlying asset can bypass these guards.

```
audit-2026-02-ethcf-swapboard/contracts/src/Swapboard.sol
57:     function createOrder(
58:         address tokenA,
59:         uint256 amountA,
60:         address tokenB,
61:         uint256 amountB
62:     ) external nonReentrant returns (uint256 orderId) {
63:         if (tokenA == address(0)) revert ZeroAddress();
64:         if (tokenB == address(0)) revert ZeroAddress();
65:         if (amountA == 0) revert ZeroAmount();
66:         if (amountB == 0) revert ZeroAmount();
67:  >>>    if (tokenA == tokenB) revert SameToken();
68:         if (tokenA.code.length == 0) revert NotAContract(tokenA);
69:         if (tokenB.code.length == 0) revert NotAContract(tokenB);

--------- skipped -------

99:         emit OrderCreated(orderId, msg.sender, tokenA, amountA, tokenB, amountB);
100:     }


150:     function createOrderWithEth(
151:         address tokenB,
152:         uint256 amountB
153:     ) external payable nonReentrant returns (uint256 orderId) {
154:         if (msg.value == 0) revert ZeroETH();
155:         if (tokenB == address(0)) revert ZeroAddress();
156:         if (amountB == 0) revert ZeroAmount();
157:  >>>    if (tokenB == weth) revert SameToken();
158:         if (tokenB.code.length == 0) revert NotAContract(tokenB);

----- skipped -----

176:         emit OrderCreated(orderId, msg.sender, weth, msg.value, tokenB, amountB);
177:     }
```

Recommended Mitigation: No mitigation needed becasue of no impact. Just documentation / info.

ETHCF: Acknowledged; by design.

7.2.2 Use named mappings parameters to explicitly indicate the purpose of keys and values

Description: Use named mappings parameters to explicitly indicate the purpose of keys and values:

```
Swapboard.sol
39:    mapping(uint256 => Order) public orders;
```

ETHCF: Fixed in commit 572f3c5.

Cyfrin: Verified.

7.2.3 Use read-then-increment in `Swapboard::createOrder, createOrderWithEth`

Description: Use read-then-increment in `Swapboard::createOrder, createOrderWithEth`:

```
-       orderId = nextOrderId;
-       unchecked {
-           ++nextOrderId;
-       }
+       unchecked { orderId = nextOrderId++; }
```

ETHCF: Fixed in commit 572f3c5.

Cyfrin: Verified.

7.3 Gas Optimization

7.3.1 Use `ReentrancyGuardTransient` instead of `ReentrancyGuard` for faster `nonReentrant` modifiers

Description: Use ReentrancyGuardTransient instead of `ReentrancyGuard` for faster `nonReentrant` modifiers:

```
Swapboard.sol
import {ReentrancyGuard} from "@openzeppelin/contracts/utils/ReentrancyGuard.sol";
///      - Reentrancy protected via OpenZeppelin ReentrancyGuard
contract Swapboard is ISwapboard, ReentrancyGuard {
```

ETHCF: Fixed in commit 572f3c5.

Cyfrin: Verified.

7.3.2 Don't initialize `received` to default value in `Swapboard::createOrder`

Description: Don't initialize `received` to default value in `Swapboard::createOrder`:

```
-       uint256 received;
        unchecked {
-           received = balanceAfter - balanceBefore;
+           uint256 received = balanceAfter - balanceBefore;
-       }
        if (received != amountA) {
            revert BalanceMismatch(amountA, received);
        }
+       }
```

ETHCF: Fixed in commit 572f3c5.

Cyfrin: Verified.

7.3.3 Cache storage to prevent identical storage reads

Description: In EVM reading from storage is expensive; cache storage to prevent identical storage reads when known values can't change:

- `fillOrder, cancelOrder` - cache `order.maker`
- `fillOrderWithEth` - cache `order.maker, order.amountB`
- `cancelOrderUnwrap, fillOrderUnwrap` - cache `order.maker, order.amountA`

ETHCF: Fixed in commits 572f3c5, 3a9e06e.

Cyfrin: Verified.

7.3.4 When sending ETH, use `SafeTransferLib::safeTransferETH` instead Of Solidity `call`

Description: When sending ETH, use SafeTransferLib::safeTransferETH instead Of Solidity `call` which is more gas efficient:

```
Swapboard.sol
205:        (bool success,) = payable(order.maker).call{value: order.amountA}("");
227:        (bool success,) = payable(msg.sender).call{value: order.amountA}("");
```

Another alternative "stand-alone" pattern which is also more efficient and prevents "return-bomb" style attacks is for example in `cancelOrderUnwrap`:

```
-       (bool success,) = payable(maker).call{value: amountA}("");
+       bool success;
+       assembly {
+           success := call(gas(), maker, amountA, 0, 0, 0, 0)
```

```
+       }
        if (!success) revert ETHTransferFailed(maker);
```

ETHCF: Fixed in commit 3a9e06e using the alternative "stand-alone" pattern.

Cyfrin: Verified.

7.3.5 Don't cache `calldata` array length in `Swapboard::getOrders`

Description: It is more gas-efficient to not cache `calldata` array length in `Swapboard::getOrders`.

ETHCF: Fixed in commit 572f3c5.

Cyfrin: Verified.

7.3.6 Better storage packing in `struct Order` reduces gas cost of all `create, fill, cancel` operations

Description: Since `Order::maker, active` are frequently written and read together, in struct `Order` it is more efficient to pack them into the same slot:

```
    struct Order {
        address maker;
+       bool active;
        address tokenA;
        uint256 amountA;
        address tokenB;
        uint256 amountB;
-       bool active;
    }
```

Then in `Swapboard::fillOrder, cancelOrder, fillOrderWithEth, cancelOrderUnwrap, fillOrderUnwrap` use this format to read them both in one SLOAD:

```
-       address maker = order.maker;
+       (address maker, bool active) = (order.maker, order.active);
        if (maker == address(0)) revert OrderNotFound(orderId);
-       if (!order.active) revert OrderNotActive(orderId);
+       if (!active) revert OrderNotActive(orderId);
```

This also reduces the gas costs of creating orders since it removes one storage write with no required code changes to those functions.

ETHCF: Fixed in commit 82f9b49.

Cyfrin: Verified.